

DRAGON BALL ROBLOX GAME

Combat Master Plan & Research

Original IP · Robux-monetized · Final-Stand-style PvP + PvE

Noea-style transformation wheel · PC-first, all devices target

TITLE BLOCK

Project	DB Roblox Combat Plan
Doc	Master Plan v1
Scope	Melee combat focus
Platform	Roblox (PC-first)
Drawn	Z.ai Research
Rev	A — Initial

Part I — Executive Summary & Project Decisions

This document is the combat-focused master plan for an original-IP Dragon Ball-inspired Roblox game. It synthesizes research from the previous compendium (BT3 decompilation audit, Sparking! ZERO design blueprint, Dragon Block Noea RPG systems, canon power-level database) with new research into Sparking! ZERO and Budokai Tenkaichi 3 melee controls, Roblox combat scripting best practices, and how the top Dragon Ball Roblox games build their feel. The goal is a single source of truth for the melee combat system before any code is written in Studio.

The user has made seven foundational decisions that gate every system below. They are recorded here so every subsequent design choice can be checked against them. Where a decision has an engineering implication that is not obvious, it is called out in the right-hand column.

1.1 The seven locked decisions

#	Decision	Engineering implication
1	Original IP (e.g. Goku -> "Son Wukong"). Renames like these must be original. Mechanics + power-level math transfer (not copy-paste).	
2	Full roster scope eventually, but start with Daizenshuu SKPL5 roster mandatory from day 1 — adding chars must not touch engine.	
3	Final-Stand-style: PvP AND co-op. PvP is the server-authoritative + client-side hit detection + ping-bounded lag compensation. Roaming.	
4	All devices (PC/PS/Xbox/mobile), but PC-first and always prioritized. No abstract away device. Mobile cannot do motion inputs — needs a touch overlay.	
5	Continue planning before building in Studio. This document IS that plan. No Studio scaffolding yet.	
6	Manga + Daizenshuu power levels (widest reasonable range) is the universal tag every value [C]/[D]/[!]. User edits inline.	
7	Noea-style transformation wheel: God Ki Tier 1, Base form, Super Saiyan 1, Super Saiyan 2, Super Saiyan 3, Super Saiyan 4, Super Saiyan God, Super Saiyan God Super Saiyan, Super Saiyan Blue, Super Saiyan Blue Kai, Super Saiyan Blue Kai Max, Super Saiyan God Super Saiyan (Ultimate) (Manga).	

1.2 The 6 starting characters (most popular — proposed)

Roster selection is the user's call, but these six are the most-recognized across the franchise and give a balanced spread of archetypes (rush-down, zoner, balanced, heavy, technique, god-tier). Each will get an original-IP rename.

Canon name	Original-IP placeholder	Archetype	Why included
Goku	Son Wukong	Balanced all-rounder	Franchise face. Every form tier (base/SSJ/SSJ3/UI).
Vegeta	Prince Vega	Heavy zoner	Rival. Final Flash = beam archetype. Pride Counter unique.
Gohan	Han	Technique / potential	Potential Unleashed mechanic = Power Release showcase.
Piccolo	Ma Junior	Reach / zoner	Namekian race (regen, stretch limbs). Non-Saiyan archetype.
Frieza	Lord Kold	Zoner / transformer	5-form transformation ladder. Planet-destroyer fantasy.
Broly	Brol	Heavy grappler	Enormous, screen-filling. Grappler archetype + raid boss.

User: confirm or swap any of these six before we design their movelists.

Part II — Sparking! ZERO & BT3 Melee Research

Before designing our melee, we need the exact control architecture of the games we are emulating. The primary sources are the official Bandai Namco Beginner's Guide and Combos & Features page, plus IGN's Advanced Combat Tips guide (Laura-May Randell et al., Oct 2024). All citations are verified against these sources; line-level claims are quoted.

2.1 The two-button melee core

Sparking! ZERO melee rests on two attack buttons and one dash button. On PlayStation these are Square (light / rush), Triangle (heavy / smash), and Cross/X (dash). On Xbox: X, Y, A. The depth comes from how these combine with directional inputs, spacing, and timing — NOT from a large button count. This is the single most important design lesson: a small input surface with deep combination rules beats a large button surface every time, and it is the only way mobile parity is achievable.

Button (PS5)	Button (Xbox)	Name	Function
Square	X	Rush Attack	Light melee. Press 1-4 times for a Rush Chain. Auto-combo on mash.
Triangle	Y	Smash Attack	Heavy melee. Hold to charge for guard-break + knockaway. Also the Rush Chain finisher.
X	A	Dash / Step	Context-sensitive: quick step at close range, quick dash at long range, Dragon Dash at long range.
Circle	B	Blast / Lightning	Ki blast at range. Lightning Attack ender after a knockaway pursuit.
R1+Dash	RB+Dash	Dragon Dash	Ki-costing long-distance dash. Dash again mid-flight = Z-Burst (zip behind, costs 1 Ki).

2.2 The Rush Chain system (this is the "Square Square Triangle" the user asked about)

A Rush Chain is Square pressed 1-4 times, then Triangle as the finisher. The number of Squares before the Triangle determines which finisher plays. This is EXACTLY the system the user described ("square square triangle = gut punch, square square square triangle = something else"). From the official Bandai Combos & Features page (Aug 2024): "Rush Chain combos can be activated by pressing [Square/X] (up to 4 times maximum) + [Triangle/Y]. The longer the combo is, the bigger the damage dealt."

Verified Rush Chain variants (Goku reference, most characters follow this pattern):

Input	Finisher name	Animation	Use case
S+T	Gut Punch	Quick gut strike, short hitstun	Fastest confirm; leads into pursuit
SS+T	Jump Kick	Launcher, sends enemy upward	Anti-air starter; air combo route
SSS+T	Sweep / Lift	Sweep or lift-strike, knockdown	Hard knockdown; okizeme pressure
SSSS+T	Heavy Smash	Charged knockaway, wall-bounce	Max damage; ends in knockaway for pursuit
(any)+hold T	Smash Charge	Charged Triangle, guard-break	Punishes guard-hold during chain gaps

Critical rule from IGN: "You can't repeat the same rush chain twice in one combo, as it will make it drop." This forces variety — you cannot loop S+T forever. A real combo links different finishers: gut punch -> jump kick -> sweep -> heavy smash -> pursuit. This is the skill ceiling.

Second critical rule from IGN: "There are gaps between rush attacks (and most rush chains) that let the opponent hold guard while they're getting hit in order to block a rush chain." So mashing a chain is countable. The attacker's answers are (a) charge the finisher to break the guard, or (b) press R1/RB to step to their side and continue. This is the defensive-offensive tension.

2.3 The knockaway pursuit system (what happens AFTER you hit them)

This is what makes Sparking! ZERO feel like Dragon Ball. From IGN: "One of the things that sets Dragon Ball Sparking! Zero apart from other arena fighters is that when you knock an enemy soaring away at 100 miles per hour, the combo isn't over. You have several different follow-ups." After ANY knockaway (rush-chain finisher,

charged smash, reversal hit), the attacker presses one of:

Button	Move	Effect
X / A	Dragon Homing	Teleport to the flying enemy, continue the combo. Each character has 2-3 (UI Goku has 4).
Square / X	Dragon Smash	Re-knock them away in a new direction. Resets the pursuit counter.
Triangle / Y	Vanishing Attack	Teleport BEHIND the enemy and strike. Opens pursuit.
Circle / B	Lightning Attack	Slam them into the ground. End-of-combo finisher. Does NOT count as a pursuit.

After landing a pursuit, Blasts (Kamehameha-class specials) are "nearly guaranteed" (IGN). So the full damage route is: rush chain -> knockaway -> pursuit -> pursuit -> lightning slam OR blast. This is the spine of our combat loop.

2.4 The Vanishing Assault (the "vanish behind" the user asked about)

This is the mechanic the user specifically named: "if two characters are about the same level and one is going to land a heavy, you can vanish it and appear behind the other person." In Sparking! ZERO this is Vanishing Assault — Square+X (PS5) or X+A (Xbox) simultaneously. From IGN: "If you're close, you'll teleport behind your opponent and knock them away, opening them up for a pursuit attack. If you're at mid-range, you'll close the distance by teleporting in front of them with a strike. It costs a full bar of meter to use."

Key design points for our Roblox translation:

- **Cost:** 1 full Ki bar. Not free. This is the resource gate that stops vanish-spam.
- **Range-conditional:** close = behind + knockaway; mid = in-front + strike. We replicate this with a distance check on activation.
- **Invuln frames:** the vanish itself is invulnerable — you disappear before the heavy lands. This is the "vanish the heavy" read.
- **Punish window:** if you vanish and DO NOT act, you are vulnerable. The vanish is an opener, not a free escape.
- **Power-level gate:** the user wants this only usable "if two characters are about the same level." We implement this as: vanish costs +50% Ki if defender PL is >2x attacker PL, and is BLOCKED if defender PL is >10x (you cannot vanish what you cannot perceive). This is the canon-faithful rule (Krillin cannot vanish Beerus).

2.5 BT3 lineage (the same DNA, verified in the decompilation)

The previous compendium audited sles_549.45_decompiled.c (Budokai Tenkaichi 3 PAL, 292,589 lines, 7,610 functions) and confirmed the same architecture: a 4-layer input pipeline (raw pad read -> input translation -> script-VM skill dispatch -> execution primitives), a 1,311-line ray-vs-triangle collision function (FUN_00232478) that returns a struck-face index for per-limb reaction dispatch, and an embedded bytecode script VM at line 6872 that makes character moves DATA not CODE. Sparking! ZERO is made by Spike Chunsoft, the direct successor of Spike (BT3's developer) — same lineage, same architecture philosophy. The Rush Chain + pursuit + vanish system above is the modern evolution of BT3's script-driven movesets.

Part III — Our Melee Combat Design

This is the core of the document. The melee system is designed to be (a) deep enough to reward practice, (b) simple enough to fit on a mobile touchscreen without motion inputs, and (c) faithful to the Sparking! ZERO / BT3 feel. Every mechanic below is mapped to a Roblox implementation in Part V.

3.1 Input model (PC-first, mobile-ready)

We do NOT use motion inputs (QCF/HCB). They are unplayable on mobile and unnecessary — Sparking! ZERO proves a two-button + modifier system carries full fighting-game depth. Our inputs:

Input (PC)	Input (mobile)	Action	Notes
Left click (M1)	Tap left side	Rush (light)	Press 1-4x for Rush Chain. Tap+hold = Smash charge.
Right click (M2)	Tap right side	Smash (heavy)	Direct heavy. Hold to charge for guard-break.
Space	Dedicated button	Jump / Fly	Tap to jump, hold to ascend, double-tap to fly.
Shift	Dedicated button	Guard	Hold to block. Drains Stamina. Breakable.
Q	Dedicated button	Vanish	Costs 1 Ki bar. Behind-enemy teleport. PL-gated.
E	Dedicated button	Counter	Revenge Counter when in hitstun (1 Skill Count).
F	Dedicated button	Ki Blast	Ranged. Holds to charge a beam.
R	Dedicated button	Dash / Step	Context: step at close range, dash at range, Dragon Dash with modifier.
1 / 2 / 3	Wheel button	Transform	Opens the 3-axis transformation wheel (Part VII).
Tab	Dedicated button	Sparking Mode	Full gauge + 1 Skill Count = free Ki + Ultimate.

Mobile uses a "two-zone tap + ability buttons" scheme. Left half of screen = light/heavy tap combo (tap count = rush chain position, long-press = smash). Right half = ability buttons (Vanish, Counter, Ki, Dash, Transform, Sparking). This is the proven mobile-fighter layout (used by Marvel Contest of Champions, Dragon Ball Legends). No motion inputs anywhere.

3.2 The Rush Chain system (our version)

Direct translation of Sparking! ZERO's Rush Chain. Light (M1) pressed N times, then Heavy (M2) as the finisher. N determines the finisher. The finisher is the only frame that consumes a "chain slot" — you cannot repeat the same finisher twice in one combo string.

Input	Finisher	Startup	Active	Recovery	On hit	Cancel into
M1 (x1) + M2	Gut Punch	4f	3f	10f	+4 (light hitstun)	Pursuit, Vanish, Blast
M1 (x2) + M2	Rising Knee	6f	3f	14f	Launcher (air combo)	Pursuit, Air chain
M1 (x3) + M2	Sweep	8f	4f	16f	Knockdown	Okizeme only (no pursuit)
M1 (x4) + M2	Heavy Smash	10f	5f	20f	Knockaway (wall-bound)	Pursuit x2, Blast
M2 (hold)	Smash Charge	20f+	5f	24f	Guard-break + knockaway	Pursuit, Vanish, Blast
M2 (raw)	Quick Heavy	7f	3f	15f	-4 (slight disadr.)	Nothing (poke use)

Frame counts at 60 FPS. "On hit" = frame advantage on hit. "+" means attacker recovers first.

The no-repeat rule: each finisher can appear at most ONCE per combo. A combo "Gut Punch -> Rising Knee -> Heavy Smash -> Pursuit -> Rising Knee" is INVALID (Rising Knee repeats) and the second Rising Knee will whiff. This forces combo variety and is the skill ceiling.

3.3 The pursuit / knockaway system

After any knockaway finisher (Heavy Smash, Smash Charge, gut-punch-into-pursuit), the attacker has a 0.4s window to input a pursuit. Each character has a Pursuit Count (default 2, fast characters 3, UI-tier 4). Pursuit options:

Input	Move	Effect	Pursuit cost
R (toward enemy)	Dragon Homing	Teleport to enemy, continue combo	1 pursuit
M1	Dragon Smash	Re-knock away, new direction	1 pursuit
M2	Vanishing Strike	Teleport behind, strike	1 pursuit
F (hold)	Lightning Slam	Slam to ground. End of combo.	0 (does not count)
F (Ki blast)	Blast Confirm	Guaranteed blast after a pursuit hit	0 (Ki cost only)

Design rule: Blasts after a pursuit are "nearly guaranteed" (IGN). This means the blast's startup is treated as a combo extender, not a standalone move, when fired within 0.2s of a pursuit connect. This is the canon "Goku Kamehamehas after knocking Broly away" beat.

3.4 The vanish system (PL-gated, Ki-costed)

Direct translation of Vanishing Assault with the user's power-level gate added. Vanish (Q) teleports the player behind the enemy and strikes. Mechanics:

- **Cost:** 1 full Ki bar (60 Ki out of 100 max).
- **Activation:** 3-frame startup (invulnerable frames 4-8), teleport, strike active frames 9-12, recovery 13-22.
- **Range:** activates within 25 studs (close). Beyond 25 studs, becomes a "Vanish Approach" (teleport in-front + strike, no behind).
- **Power-level gate (canon-faithful):** if defender.effectivePL > 2x attacker.effectivePL, Ki cost doubles to 2 bars. If > 10x, vanish is BLOCKED with a "too fast to perceive" message. This is why Krillin cannot vanish Beerus.
- **Counter-play:** a vanished player is vulnerable for 4 frames after the strike. A defender who reads the vanish can Guard (free) or Super Counter (timing) the strike.
- **Stamina gate:** vanish sets Stamina to 0 for 1.5s. Cannot vanish twice in a row. This stops vanish-spam.

3.5 The 5-counter ladder (defense is the skill ceiling)

From the compendium's Step 4. The tiered risk ladder is what makes defense non-binary.

Counter	Input	Cost	Beats	Risk
Guard	Shift (hold)	Stamina drain	General pressure	Guard-breakable by Smash Charge
Super Perception	E (timed)	Free vs melee / 2 Skill Count	Rush attacks, blasts	Baitable; whiffs on a Dash approach
Z-Counter	E (timed, in hitstun)	1 Skill Count	Vanishing Assault	Mistime = take the hit
Revenge Counter	E (in hitstun)	1 Skill Count	Being combo'd (panic escape)	Predictable; baitable by free Perception
Super Counter	Shift+M2 (precise)	None	Almost any attack (from behind or fast hit)	Timing in the game (4f window)

3.6 The full damage route (putting it together)

A textbook max-damage combo from neutral:

Step	Input	Move	Resource state
1	Approach with R (Dragon Dash)	Close distance	Ki -20 (dash)
2	M1 M1 M2	Rush Chain x2 + Rising Knee (launcher)	Ki +5 (on hit)
3	(auto air) M1 M1 M2	Air Rush Chain + Gut Punch	Skill Count +1 (on hit)
4	R	Dragon Homing pursuit	Pursuit 1/2
5	M2	Vanishing Strike (re-knockaway)	Pursuit 2/2
6	F (hold)	Blast Confirm (Kamehameha-class)	Ki -40
7	(optional) Tab + F	Sparking Mode -> Ultimate Blast	Sparking gauge + 1 Skill

Total: ~3.5 seconds, ~40% HP if all hits connect, costs ~60 Ki + 1 Skill Count + full Sparking. This is a "round-changing" combo, not a bread-and-butter. Bread-and-butter is steps 1-3 only (~12% HP, Ki-neutral). The depth is in READING which route the defender gives you.

Part IV — Canon Power Level Master List (Manga + Daizenshuu)

The user requested manga + Daizenshuu sources, with the ability to adjust as we go. Every value is tagged [C] = manga on-panel, [D] = Daizenshuu 7, [S] = scaling-derived from canon multipliers, [!] = contested (multiple sources disagree). The user should edit this table directly. These values feed the log-ratio damage formula: $\text{ratioScale} = \log_{10}(\text{attacker.effectivePL} / \max(\text{defender.effectivePL}, 1) + 9)$.

4.1 Saiyan Saga (Raditz -> Vegeta)

Character	Form	Power Level	Source	Notes
Farmer	-	5	[C]	Reference baseline (manga panel)
Son Wukong (Goku)	Base	416	[C]	Raditz fight, weighted
Son Wukong (Goku)	Kamehameha	924	[C]	Raditz fight, concentrated
Piccolo (Ma Jr.)	Base	408	[C]	Raditz fight
Piccolo (Ma Jr.)	Makankosappo	1,440	[C]	Raditz fight (Special Beam Cannon)
Raditz	Base	1,500	[C]	Scouter reading on panel
Krillin	Base	1,770	[D]	Daizenshuu 7, post-Saiyan training
Tien	Base	1,830	[D]	Daizenshuu 7
Yamcha	Base	1,480	[D]	Daizenshuu 7
Son Wukong (Goku)	Kaio-ken x4	1,664	[S]	416 x 4 (manga-stated multiplier)
Nappa	Base	4,000	[!]	Daizenshuu vs manga showings; contested
Vegeta (Vega)	Base	18,000	[C]	Scouter reading, manga-stated
Vegeta (Vega)	Oozaru	~180,000	[S]	x10 Oozaru multiplier (canon)

4.2 Namek / Frieza Saga

Character	Form	Power Level	Source	Notes
Cui	-	18,000	[C]	Equal to Vegeta base
Dodoria	-	22,000	[D]	Daizenshuu 7
Zarbon	Base	23,000	[D]	Daizenshuu 7
Zarbon	Transformed	30,000	[D]	Daizenshuu 7
Vegeta (Vega)	Post-Zenkai	30,000	[S]	Scaling from Zarbon fight
Ginyu	Base	120,000	[C]	Scouter-stated
Frieza (Kold)	1st	530,000	[C]	Stated by Frieza himself
Frieza (Kold)	2nd	~1,000,000	[D]	Daizenshuu 7
Frieza (Kold)	3rd	~2,000,000	[S]	Scaling
Frieza (Kold)	Final (50%)	60,000,000	[D]	Daizenshuu 7
Frieza (Kold)	Final (100%)	120,000,000	[D]	Daizenshuu 7
Son Wukong (Goku)	Base (post-Zenkai)	3,000,000	[S]	Scaling from Frieza fight
Son Wukong (Goku)	Kaio-ken x10	30,000,000	[S]	3M x 10
Son Wukong (Goku)	Kaio-ken x20	60,000,000	[S]	3M x 20, matches Frieza 50%
Son Wukong (Goku)	SSJ	150,000,000	[D]	Daizenshuu 7 (x50 multiplier)

4.3 Cell Saga

Character	Form	Power Level	Source	Notes
Trunks (future)	SSJ	~150,000,000	[S]	Matches SSJ Goku on Namek arrival
Android 19/20	-	~10,000,000	[!]	V-Jump-derived; contested
Android 17	-	~300,000,000	[!]	V-Jump; contested
Android 18	-	~280,000,000	[!]	V-Jump; slightly below 17
Imperfect Cell	-	~45,000,000	[S]	Scaling
Semi-Perfect Cell	-	~90,000,000	[S]	Scaling
Vegeta (Vega)	SSJ (post-RoSAT)	~180,000,000	[S]	Scaling; > Semi-Perfect
Perfect Cell	Base	~900,000,000	[S]	Scaling
Perfect Cell	Suppressed	~1,200,000,000	[S]	vs Goku full-power
Son Wukong (Goku)	FP SSJ	~1,200,000,000	[S]	Matches suppressed Perfect Cell
Han (Gohan)	SSJ2	~1,800,000,000	[S]	x100 of ~18M base; manga > Super Perfect Cell
Super Perfect Cell	-	~1,600,000,000	[S]	Zenkaied; still < SSJ2 Gohan

4.4 Buu Saga

Character	Form	Power Level	Source	Notes
Majin Vegeta (Vega)	SSJ2	~1,800,000,000	[S]	Matches SSJ2 Gohan (Cell Games) per manga
Son Wukong (Goku)	SSJ2	~3,000,000,000	[S]	Scaling; > Majin Vegeta
Son Wukong (Goku)	SSJ3	24,000,000,000	[V]	V-Jump
Fat Buu	-	~8,000,000,000	[S]	Scaling; > SSJ2 but < SSJ3
Super Buu	-	~24,000,000,000	[S]	= SSJ3 Goku (manga equality; Goku refused to fight)
SSJ3 Gotenks	-	~30,000,000,000	[S]	> SSJ3 Goku (manga-stated by Goku)
Ultimate Han (Gohan)	-	~35,000,000,000	[S]	> SSJ3 Gotenks; > Super Buu (manga dominance)
Buutenks	-	~48,000,000,000	[S]	Briefly > Ultimate Gohan
Buuhan	-	~65,000,000,000	[S]	Gohan + Gotenks + Piccolo absorbed
Vegito (base)	-	~70,000,000,000+	[S]	Fused; beyond Buuhan
Kid Buu	-	~24,000,000,000	[S]	= SSJ3 Goku (manga equality)

4.5 God-tier (DBS — tag all as [S] or [!], NOT canon-stated)

Character	Form	Power Level	Source	Notes
Beerus	-	Unstated	[!]	God of Destruction; no canon number
Son Wukong (Goku)	SSG	~2,400,000,000,000	[S]	Scaling (x100 of SSJ3 if SSG ~ x100 SSJ3)
Son Wukong (Goku)	SSB	~12,000,000,000,000	[S]	SSG x SSJ (x50)
Golden Frieza (Kold)	-	~12,000,000,000,000	[S]	Matches SSB at launch
Son Wukong (Goku)	UI Omen	Unstated	[!]	No canon multiplier; treat as [S] x1.5 SSB
Son Wukong (Goku)	Mastered UI	Unstated	[!]	Peak; treat as [S] x2 SSB
Jiren	-	Unstated	[!]	> Mastered UI tier
Broly (Brol, movie)	SSJ	Unstated	[!]	Matches SSB Goku + Vegeta individually
Gogeta (fused)	SSB	Unstated	[!]	> Broly SSJ

God-tier numbers are speculative. The honest approach: store them as tier-relative multipliers (SSG = x100 SSJ3, SSB = x5000 SSJ3, UI = x7500 SSJ3) rather than absolute billions, and let the log-ratio formula handle the scale. The user should review and correct.

Part V — Roblox Implementation: The 4-Layer Melee Pipeline

The melee system above is implemented as BT3's proven 4-layer pipeline, Roblox-translated. This is the architecture the decompilation audit verified at line 6872 of sles_549.45. Every layer is independently testable; the data-driven Layer 3 (SKILLS table) is what makes the roster scale to 6 -> 98 characters without touching engine code.

5.1 The four layers

Layer	BT3 name	Our Roblox implementation	File location
1 — Raw input	Pad reader (FUN_00126ff8)	UserInputService + ContextActionService -> StartPlayerScripts, ClientPressed set	StartPlayerScripts, ClientPressed set
2 — Input translation	Intent-flag writer (exists, not finalized in BT3)	offer; recognizes tap-count	StartPlayerScripts, ClientPressed set
3 — Skill dispatch	Script VM (line 6872)	SKILLS ModuleScript table indexed by skillId	ReplicatedStorage, Skills (data) is the table
4 — Execution	Animation + hitbox + camera	AnimatoirPlay, RaycastHitboxV4, TweenService	ReplicatedStorage, Engine, SEVerScriptSer

5.2 The frame-data MoveDef (every melee move is one of these)

This is the data structure the SKILLS table holds for every melee move:

```
type MoveDef = {
  name: string,
  kind: "rush" | "smash" | "pursuit" | "vanish" | "blast",
  chainSlot: number?, -- 1-4 for rush-chain finishers (nil for non-chain)
  startup: number, -- frames before hitbox active
  active: number, -- frames hitbox is live
  recovery: number, -- frames after active until next action
  baseDamage: number,
  kiCost: number,
  staminaCost: number,
  hitstun: number, -- frames defender is locked
  knockback: Vector3, -- direction + magnitude
  knockaway: boolean, -- sends enemy flying (enables pursuit)
  guardBreak: boolean, -- breaks a holding guard
  cancelWindows: { -- where this move can be cancelled
    {from: number, to: number, into: {skillId}}
  },
  hitbox: {limb: string, size: Vector3}, -- which body part hits
  anim: {track: string, priority: number, fade: number},
  onHit: (attacker, defender, faceIndex) -> ()?, -- per-face reaction hook
}
```

5.3 The SKILLS dispatch table (Layer 3 — the BT3 script-VM equivalent)

This is the exact pattern the compendium's Step 9 prescribes. Moves are DATA.

```
-- ReplicatedStorage.Skills.Init (ModuleScript)
local Skills = {}

-- RUSH CHAIN FINISHERS (chainSlot 1-4)
Skills[1] = {
  name = "GutPunch", kind = "rush", chainSlot = 1,
  startup = 4, active = 3, recovery = 10,
  baseDamage = 5, kiCost = 0, staminaCost = 2,
  hitstun = 14, knockback = Vector3.new(0,0,-2), knockaway = false,
  guardBreak = false,
  cancelWindows = {{from=4, to=9, into={2,3,4,5,10}}}, -- chain into next slot or vanish
  hitbox = {limb = "RightHand", size = Vector3.new(2,2,2)},
  anim = {track = "GutPunch", priority = Enum.AnimationPriority.Action, fade = 0.05},
}
Skills[2] = { name = "RisingKnee", kind = "rush", chainSlot = 2, startup=6, active=3, recovery=14,
  baseDamage=7, kiCost=0, staminaCost=3, hitstun=18,
  knockback = Vector3.new(0,8,0), knockaway=false, -- LAUNCHER (air combo starter)
  cancelWindows = {{from=6, to=12, into={3,4,10}}},
  hitbox = {limb = "RightUpperLeg", size = Vector3.new(2,3,2)},
  anim = {track = "RisingKnee", priority = Enum.AnimationPriority.Action, fade = 0.05},
}
```

```

Skills[3] = { name = "Sweep", kind = "rush", chainSlot = 3, startup=8, active=4, recovery=16,
baseDamage=8, kiCost=0, staminaCost=4, hitstun=20,
knockback = Vector3.new(0,-1,-1), knockaway=false, -- KNOCKDOWN
cancelWindows = {}, -- no cancel; full commitment
hitbox = {limb = "RightLowerLeg", size = Vector3.new(3,1,3)},
anim = {track = "Sweep", priority = Enum.AnimationPriority.Action, fade = 0.05},
}
Skills[4] = { name = "HeavySmash", kind = "rush", chainSlot = 4, startup=10, active=5, recovery=20,
baseDamage=12, kiCost=0, staminaCost=5, hitstun=24,
knockback = Vector3.new(0,2,-12), knockaway=true, -- KNOCKAWAY (enables pursuit)
guardBreak=false, cancelWindows={{from=10,to=18,into={20,21,22}}}, -- into pursuit
hitbox = {limb = "RightHand", size = Vector3.new(3,3,3)},
anim = {track = "HeavySmash", priority = Enum.AnimationPriority.Action, fade = 0.05},
}

-- SMASH (heavy, hold to charge)
Skills[5] = { name = "SmashCharge", kind = "smash", startup=20, active=5, recovery=24,
baseDamage=18, kiCost=0, staminaCost=8, hitstun=30,
knockback = Vector3.new(0,4,-16), knockaway=true, guardBreak=true,
cancelWindows = {{from=20,to=22,into={20,21}}},
hitbox = {limb = "RightHand", size = Vector3.new(4,4,4)},
anim = {track = "SmashCharge", priority = Enum.AnimationPriority.Action, fade = 0.1},
}

-- PURSUITS (after knockaway)
Skills[20] = { name = "DragonHoming", kind = "pursuit", startup=2, active=4, recovery=8,
baseDamage=4, kiCost=10, staminaCost=0, hitstun=16,
knockback = Vector3.new(0,0,-4), knockaway=true,
cancelWindows = {{from=2,to=6,into={21,22,23}}},
-- pursuit flag: this move can only fire during a knockaway window
requires = {state = "knockawayWindow"},
anim = {track = "DragonHoming", priority = Enum.AnimationPriority.Action2, fade = 0.03},
}
Skills[21] = { name = "VanishingStrike", kind = "pursuit", startup=4, active=3, recovery=10,
baseDamage=6, kiCost=15, staminaCost=0, hitstun=18,
knockback = Vector3.new(0,3,-8), knockaway=true,
-- TELEPORT BEHIND on activation (the vanish feel)
onActivate = function(attacker, defender)
local behind = defender.CFrame * CFrame.new(0, 0, 4)
attacker:PivotTo(behind)
end,
anim = {track = "VanishingStrike", priority = Enum.AnimationPriority.Action2, fade = 0.03},
}

-- VANISH (the standalone Vanishing Assault)
Skills[10] = { name = "Vanish", kind = "vanish", startup=3, active=4, recovery=10,
baseDamage=8, kiCost=60, staminaCost=100, -- full Ki bar + full stamina drain
hitstun=16, knockback = Vector3.new(0,0,-6), knockaway=true,
onActivate = function(attacker, defender)
-- POWER-LEVEL GATE (the canon rule)
local aPL = require(game.ReplicatedStorage.Engine.Damage).effectivePL(attacker)
local dPL = require(game.ReplicatedStorage.Engine.Damage).effectivePL(defender)
local ratio = dPL / math.max(aPL, 1)
if ratio > 10 then
return false, "too_fast_to_perceive" -- BLOCKED
elseif ratio > 2 then
attacker.ki -= 60 -- extra bar (double cost)
end
-- teleport behind
local behind = defender:GetPivot() * CFrame.new(0, 0, 4)
attacker:PivotTo(behind)
attacker.involn = 8 -- i-frames during teleport
end,
anim = {track = "Vanish", priority = Enum.AnimationPriority.Action3, fade = 0.02},
}

-- DISPATCH (the script VM)
local function dispatch(skillId, attacker, target)
local skill = Skills[skillId]
if not skill then warn("[Skills] unknown", skillId) return false end
local char = require(game.ReplicatedStorage.Engine.Character).get(attacker)
if char.ki < (skill.kiCost or 0) then return false, "no_ki" end
if char.stamina < (skill.staminaCost or 0) then return false, "no_stamina" end
if skill.requires and not char:checkRequire(skill.requires) then return false, "bad_state" end
if skill.onActivate then

```

```

local ok, err = skill.onActivate(char, target)
if ok == false then return false, err end
end
char.ki -= (skill.kiCost or 0)
char.stamina -= (skill.staminaCost or 0)
char:startMove(skill, target)
return true
end

return { Skills = Skills, dispatch = dispatch }

```

5.4 The character state machine (drives every move)

A fighter is always in exactly one state. Moves are only legal in certain states.

State	Legal moves	Exit condition
idle	Rush (M1), Smash (M2), Vanish, Ki, Dash, Transform	Any input or hit
walk	Same as idle	Stop, jump, or hit
airborne	Air Rush, Air Smash, Vanish, Ki	Land or pursuit
rushChain	Next chain slot (M1), Finisher (M2), Cancel (in window)	Finisher fires or chain drops
smashCharge	Release (M2 up), Cancel (in window)	Release or cancel
hitstun	Revenge Counter (E, costs Skill), nothing else	Timer expires or counter
knockback	Nothing (waiting for attacker pursuit)	Lands or pursuit connects
knockawayWindow	Pursuit (R/M1/M2/F) — attacker only	Pursuit window (0.4s) expires
guard	Super Counter (Shift+M2), release (Shift up)	Release or guard-break
transform	Nothing (locked in cinematic)	Cinematic ends (~1.2s)
sparking	Ultimate Blast (F hold), all normal moves	10s timer

5.5 The per-limb hitbox rig (BT3 FUN_00232478 translated)

This is the single most important Roblox-specific decision. A single RootPart.Touched detector is what every bad Roblox DBZ game does, and it is why they feel bad. BT3 uses 14 per-limb hitbox faces (Face_0x0..Face_0xe) and returns the struck face index for per-face reaction dispatch. We replicate this exactly using Swordphin's RaycastHitboxV4 (devforum.roblox.com/t/raycast-hitbox-401, the community-standard melee hitbox module).

```

-- ServerScriptService.Rigging.BuildHitboxRig (ModuleScript)
local LIMBS = {
    "Head", "UpperTorso", "LowerTorso",
    "LeftUpperArm", "LeftLowerArm", "LeftHand",
    "RightUpperArm", "RightLowerArm", "RightHand",
    "LeftUpperLeg", "LeftLowerLeg", "LeftFoot",
    "RightUpperLeg", "RightLowerLeg", "RightFoot",
}

local function build(character)
    local folder = Instance.new("Folder")
    folder.Name = "Hitboxes"
    folder.Parent = character
    for i, name in ipairs(LIMBS) do
        local limb = character:FindFirstChild(name)
        if not limb then continue end
        local hb = Instance.new("Part")
        hb.Name = string.format("Face_0x%x", i - 1) -- Face_0x0 .. Face_0xe
        hb.Size = limb.Size * 1.05 -- slight pad so the limb covers
        hb.CanCollide = false
        hb.Transparency = 1
        hb.Massless = true
        hb.Parent = folder
        local weld = Instance.new("Weld")
        weld.Part0, weld.Part1 = limb, hb
        weld.C0 = CFrame.new()
        weld.Parent = hb
    end
end

```

```
-- RaycastHitboxV4 needs 2-3 attachments per hitbox for the swept ray
for j = 0, 2 do
    local att = Instance.new("Attachment")
    att.Name = "RayPoint" .. j
    att.Position = Vector3.new(0, (j - 1) * (limb.Size.Y / 3), 0)
    att.Parent = hb
end
end
return folder
end

return { build = build, LIMBS = LIMBS }
```

The struck Part's NAME is the face index. When the attacker's RightHand hitbox overlaps the defender's Face_0x5 (LeftHand), the reaction dispatcher reads "Face_0x5" and plays the left-arm hit reaction. No lookup, no switch statement, no hardcoded limb mapping. This is BT3's actual architecture (verified at line 194764 of sles_549.45) translated to Roblox one-to-one.

5.6 Hit detection authority (the Roblox-specific compromise)

Rollback netcode is NOT feasible on Roblox (see Part VI). The consensus best-practice pattern from the devforum "Client side or Server side hitboxes?" thread and the "Building Server-Truth Combat Systems" video is: CLIENT detects for feel (no input lag), SERVER validates for security. The server's validation includes a ping-bounded lag-compensation check — the hit is legal if the target was within range on the attacker's client within their ping window.

```
-- ReplicatedStorage.Engine.CombatValidator (ModuleScript, server-side)
local MAX_PING_MS = 250 -- reject hits from clients with ping > 250ms
local PING_TOLERANCE = 1.5 -- allow 1.5x the attacker's ping as the hit window

local function validateHit(attacker, defender, move, clientTime)
    -- 1. alive check
    if defender.Humanoid.Health <= 0 then return false, "dead" end
    -- 2. cooldown check (server-tracked)
    if attacker:GetAttribute("lastMoveTime") and
    os.clock() - attacker:GetAttribute("lastMoveTime") < move.recovery / 60 then
        return false, "too_fast"
    end
    -- 3. range check (lag-compensated)
    local ping = attacker:GetAttribute("ping") or 50
    if ping > MAX_PING_MS then return false, "high_ping" end
    local dist = (attacker:GetPivot().Position - defender:GetPivot().Position).Magnitude
    local maxRange = move.hitbox.size.Magnitude * PING_TOLERANCE * (1 + ping/200)
    if dist > maxRange then return false, "out_of_range" end
    -- 4. state check (defender must be in a hittable state)
    local dState = defender:GetAttribute("state")
    if dState == "invuln" or dState == "transform" then return false, "invulnerable" end
    -- passed
    attacker:SetAttribute("lastMoveTime", os.clock())
    return true
end

-- RemoteEvent handler
local hitEvent = game.ReplicatedStorage.WaitForChild("HitEvent")
hitEvent.OnServerEvent:Connect(function(attacker, defender, moveId, clientTime)
    local move = require(game.ReplicatedStorage.Skills).Skills[moveId]
    if not move then return end
    local ok, err = validateHit(attacker, defender, move, clientTime)
    if not ok then
        warn(string.format("[Combat] rejected hit from %s: %s", attacker.Name, err))
        return
    end
    require(game.ReplicatedStorage.Engine.Damage).apply(attacker, defender, move, 0)
end)
```

5.7 The log-ratio damage formula (canon power scaling)

From the compendium's Step 5. Canon PL spans 7 orders of magnitude (Farmer 5 to Vegito 70B). A linear formula makes 99% of matchups one-shot kills. The log-scaled combat-ratio keeps a 100x advantage decisive but survivable, exactly the manga's drama.

```

-- ReplicatedStorage.Engine.Damage (ModuleScript)
local math = math

local function effectivePL(char)
local base = char.basePL
local form = char.forms[char.formIdx]
local formMul = form.pLMul
local release = char.powerRelease -- 0.0 to 1.0 (Power Release throttle)
local mastery = char.mastery[char.formIdx] or 0 -- 0.0 to 1.0
local masteryMul = 1 + mastery * 0.5 -- up to +50% at full mastery
return base * formMul * masteryMul * (0.5 + 0.5 * release) -- min 50% at release 0
end

local function ratioScale(ratio)
-- compendium formula: log10(ratio + 9)
-- 1x -> 1.0, 10x -> 1.28, 100x -> 2.04, 1000x -> ~3.0
return math.log10(ratio + 9)
end

local function apply(attacker, defender, move, faceIndex)
local aPL = effectivePL(attacker)
local dPL = effectivePL(defender)
local ratio = aPL / math.max(dPL, 1)
local scale = ratioScale(ratio)
local guard = defender.guardActive and 0.5 or 0 -- 50% reduction if guarding
local typeMul = move.kind == "ultimate" and 3.0
or move.kind == "blast" and 1.5
or move.kind == "smash" and 1.2
or 1.0
local dmg = move.baseDamage * scale * typeMul * (1 - guard)
defender.Humanoid:TakeDamage(dmg)
defender:SetAttribute("state", "hitstun")
defender:SetAttribute("hitstunTimer", move.hitstun)
-- per-face reaction (BT3 per-face dispatch)
require(game.ReplicatedStorage.Engine.Reactions).play(defender, faceIndex, move)
return dmg
end

return { effectivePL = effectivePL, ratioScale = ratioScale, apply = apply }

```

Part VI — Netcode: The Honest Roblox Reality

This is the part where the compendium's "rollback for PvP" hits a Roblox-specific wall. The user wants Final-Stand-style PvP+PvE and said PvP is the popularity driver. We need to be honest about what Roblox can and cannot do, because shipping a fighting game with bad netcode is a launch-killer.

6.1 Why rollback is not feasible on Roblox

- **No deterministic-tick API.** Roblox runs at variable framerate. You can pin your logic to `RunService.Heartbeat` with `dt` accumulation, but the engine's own character replication is not deterministic.
- **No state save/restore under 2ms.** Rollback requires snapshotting the entire sim state and re-simulating from a past frame. Roblox has no built-in; you would have to snapshot every `Character/Motor6D/animation-time/state` yourself — expensive and error-prone.
- **Client prediction fights the engine.** Roblox's network layer owns character replication. Rolling your own prediction means disabling network ownership and hand-rolling replication — a massive undertaking.

6.2 What we actually do (the proven Roblox pattern)

Server-authoritative with client-side hit detection + ping-bounded lag compensation (the pattern from Part V.6). This is what every successful Roblox fighting game uses. It is not rollback, but it feels acceptable up to ~80-120ms ping. For PvE and co-op (the Noea model), it is fine — Minecraft combat is server-authoritative too.

Mode	Netcode	Feel	Roblox feasibility
PvE (saga, bosses, raids)	Server-authoritative, no prediction needed	Excellent (no ping matters)	Native — just use <code>RemoteEvents</code>
Co-op (up to 4 players)	Server-authoritative + light client prediction	Good up to 150ms	Standard pattern
PvP (1v1 ranked)	Server-authoritative + client-side hit detection	Acceptable up to 120ms	Achievable but honest about limits
PvP (casual FFA)	Same as 1v1 but more chaotic	Acceptable	Achievable

The honest position: PvP on Roblox will never feel as good as GGPO-backed fighters (Street Fighter 6, MK1, Killer Instinct). But Dragon Ball Z: Final Stand and Dragon Blox Ultimate have proven that server-authoritative PvP is "good enough" for millions of players. We match that bar.

6.3 The mitigation levers (making server-authoritative feel smoother)

- **Client-side hit detection:** the attacker's client registers the hit immediately (no round-trip), sends to server for validation. Visual feedback is instant.
- **Ping-bounded tolerance:** the server accepts a hit if the target was within range on the attacker's client within 1.5x their ping. This compensates for the attacker seeing a slightly-old defender position.
- **Move-and-cancel prediction:** the client predicts its own move startup locally; the server only corrects if the move was illegal. Cancellation windows feel responsive even at 100ms.
- **Animation priority on Action layer:** attacks play instantly on the client (Action priority overrides Movement). The server confirms damage 50-100ms later. The visual is always in sync with input.
- **Region-locked matchmaking:** only match players within 120ms ping of each other. This is the single biggest feel lever.

Part VII — The Noea-Style Transformation Wheel

The user wants the wheel exactly as Noea has it: three axes — God Ki on one side, race transformations on another, power-boosting techniques (Kaioken) on the third. Each axis is a separate progression track. Forms within an axis can stack (e.g. SSG -> SSB). Forms across axes interact by design rules (e.g. Kaioken CAN stack on top of SSB, but at high Ki-drain).

7.1 The three axes

Axis	What it is	Example forms	Resource
God Ki (divine)	God-tier transformations, endgame	SSG, SSB, SSB Kaioken, UI Omen, Mastered UI	Ki Drain, Upkeep + Mastery curve
Race forms (innate)	The character's race-specific layer	Saiyan: SSJ/SSJ2/SSJ3/SSJ4; Namekian: Giant Orange	Ki/Orange Upkeep + Skill Count to activate
Power-boost (technique)	Learnable multipliers, stackable	Kaioken (x4/x10/x20), Potential Unleashed	Stamina Drain, Upkeep cost at high tiers

7.2 Stacking rules (what can combine)

Combo	Allowed?	Effect	Cost
SSB + Kaioken	YES	SSB Kaioken (canon)	Massive Ki + Stamina drain; HP drains at
SSJ3 + Kaioken	YES (risky)	Body-damaging; HP drains fast	Both upkeeps + HP
UI + Kaioken	NO	UI requires empty mind; Kaioken is the opposite	
Potential Unleashed + Kaioken	YES	Ultimate Kaioken (fan-canon)	Stamina + HP drain
God Ki + Race form	NO (one replaces SSB other)	SSS (replaces SSJ); you are in ONE form at a time	
Power-boost stack on ANY	YES (except UI)	Kaioken layers on whatever form you are in	Additive drain

7.3 The wheel UI

On press of the Transform key (1/2/3 on PC, wheel button on mobile), a radial wheel appears with three segments. The player drags toward the desired form. Release activates. The wheel pauses the game for 0.3s (Sparking-style) so the activation is deliberate, not panic.

```
-- ReplicatedStorage.UI.TransformWheel (LocalScript, client)
local WHEEL = {
  -- segment, angle-start, angle-end, axis, forms[]
  {seg="GodKi", a1=0, a2=120, axis="god", forms={"SSG","SSB","UIOmen","MasteredUI"}},
  {seg="Race", a1=120, a2=240, axis="race", forms={"Base","SSJ","SSJ2","SSJ3","SSJ4"}},
  {seg="Boost", a1=240, a2=360, axis="boost", forms={"KaiokenX4","KaiokenX10","KaiokenX20","PotentialUnleashed"}},
}
-- drag toward a segment, then toward a form within the segment
-- release = activate; the server validates (Skill Count, Ki, mastery unlock)
-- 0.3s game-pause during selection (Sparking-style)
```

7.4 Mastery curve (the Noea progression layer)

Each form has a Mastery value (0.0 to 1.0) earned by using the form. Mastery lowers Ki-upkeep and raises the stat multiplier. A fully-mastered form reaches x1.5 stats at x0.75 upkeep. Some forms become free to hold at full mastery. This is the long-term grind that keeps players in the game (the Final Stand retention loop).

Part VIII — Animation Pipeline (How to Animate the Moves)

Combat feel is 50% systems and 50% animation. A perfectly-coded Rush Chain feels terrible if the punch animation is floaty. This section covers the specific animation techniques for each move type, grounded in Roblox's Animation Editor capabilities and the community's proven practices.

8.1 Rig setup (R15 + Motor6D)

Use R15 (not R6 or R15-appended). R15 has the joint granularity needed for combat: separate UpperArm/LowerArm/Hand and UpperLeg/LowerLeg/Foot. Each joint is a Motor6D that the Animation Editor can key. For custom limb hits (e.g. a tail, a third eye), add extra Motor6Ds to the rig (devforum.roblox.com/t/how-to-animate-your-tool-using-motor6d-.../3026706 covers the pattern).

8.2 Animation priority layers (the layered, not flat, rule)

From the compendium's Step 8. Roblox supports this natively via AnimationPriority. Set every animation's priority explicitly — never leave it at Core.

Priority	Used for	Examples
Core (0)	Never use	-
Movement (1)	Locomotion blend tree	Idle, Walk, Run, Fly, Dash
Action (2)	Discrete attacks	Rush Jab, Gut Punch, Smash, Ki Blast charge
Action2 (3)	Reactions + pursuits	High-hit, Low-hit, Gut-hit, Back-hit, Knockdown, Getup, Dragon Homing
Action3 (4)	Transformation + Ultimate	SSJ transform cinematic, Kamehameha fire, Ultimate Blast
Action4 (5)	Cutscene-locked	Saga dialogue, special finishers

Critical: Action (2) plays OVER Movement (1). So a punch overlays the walk cycle — the legs keep walking while the arms punch. This is what makes a fighter feel responsive instead of "stop to attack". For the rush chain, each finisher is a separate Action animation; the Movement layer keeps running underneath.

8.3 How to animate a punch (frame-by-frame)

A clean Gut Punch at 60 FPS export is 22 frames (0.37s) — close to BT3's light-attack timing. Three keyframes minimum, four for quality:

Frame	Phase	Pose	Notes
0	Idle/entry	Guard up, weight centered	Transition from Movement layer
4	Wind-up	Fist chambers at hip, torso rotates back 15deg, weight shifts to back foot	Busiest animation frame — the "weight" frame
8	Extension	Fist fully extended, torso rotates forward 20deg, weight shifts to front foot	THE HIT FRAME — hitbox active here (frames 8-10)
14	Recovery	Arm returns to guard, weight rebalances	Cancel window opens frame 11 (can chain into next)
22	Exit	Back to idle-ready	If not cancelled, returns to Movement

The single most important frame is the wind-up (frame 4). Without it, the punch looks like a twitch. The wind-up is the "tell" that sells the weight. BT3's punches have a 2-3 frame wind-up even on jabs; this is why they feel heavy.

8.4 How to animate the Dragon Throw (grappling throw)

Paired two-rig animation. The thrower and the victim play synchronized animations:

Frame	Thrower	Victim
0-8	Reach-grab (both hands extend, lean forward)	Stagger-back (off-balance, arms flail)
8-12	Grip-and-lift (pull victim to chest, then overhead)	Be-lifted (body goes limp, rotates to upside-down)

Frame	Thrower	Victim
12-18	Slam (drive victim down into ground with both arms)	Be-slammed (spine-first impact, bounce)
18-26	Recover (release, step back, return to guard)	Knockdown bounce -> prone

Implementation: the server welds the victim to the thrower's RightHand on frame 8 (Motor6D weld), both clients play their animation, the weld is destroyed on frame 18 (slam release) and the victim is given downward velocity + the knockdown state. The hitbox activates on frame 18 (the slam) — that is the damage frame.

8.5 How to animate the Kamehameha (charge + fire)

Two-phase animation paired with VFX (Beam instance + TweenService):

Frame	Phase	Character animation	VFX
0-18	Charge (cup hands to hip)	Cup hands to hip, lean back, eyes close then back open	Beam emitter at hands grows; aura flares
18-22	Thrust (arms forward)	Arms thrust forward, palms-out, lean forward	Beam Width0 tweens 0 -> full over 4 frames
22-112	Fire (hold)	Arms locked, slight tremor	Beam active; raycast damage each frame
112-136	Recover (arms down)	Arms drop, stagger back	Beam Width tweens full -> 0 over 6 frames

The damage ray is NOT the visible Beam — the Beam is cosmetic. A separate workspace:Raycast runs each frame during the active window (22-112) along the beam direction. FilterDescendantsInstances = {attacker} so you do not self-hit. devforum.roblox.com/t/how-would-i-make-a-beam-vfx-for-a-certain-attack/2934318 covers the Beam+TweenService pattern; the YouTube "Advanced Roblox Skill Framework Part 4 (Kamehameha Beam)" tutorial covers the full server-authoritative version.

8.6 How to animate a transformation

Three-phase sequence, ~1.2s total:

Phase	Duration	What happens
Wind-up	0.8s	Aura particles spawn (charge color), character plays "powering up" anim (scream pose, fists clenched), c
Flash	0.05s	Single bright frame (white screen flash), burst of particles, camera hard shake, hair mesh swaps to new f
Hold	0.35s	New form idle kicks in (Action3 priority), aura settles to new color, stat multipliers apply, Ki-upkeep loop

The mesh-swap (not tween) approach is what most DB Roblox games use because SSJ hair geometry is structurally different from base hair — you cannot tween spikes into existence. Swap the MeshId on the Hair part at the flash frame. TweenService handles the Color transition over 0.2s so the color does not pop.

8.7 How to animate story cutscenes

Roblox has no native sequencer. Cutscenes are scripted event chains. The pattern from kitsblox.com "How to Make Smooth Camera Cutscenes in Roblox" (Mar 2026) and the devforum "Tweening Multiple Camera Points" thread (Nov 2019):

- **Camera:** set Camera.CameraType = Scriptable, then TweenService on CFrame through waypoints. Use Catmull-Rom interpolation (not linear) so the camera does not kink at waypoints.
- **Dialogue:** TextLabel with a typewriter effect (per-character spawn loop), AudioStream for text blips.
- **Event chain:** a Cutscene Module that takes a list of {time, action} pairs and runs them on a timeline.
- **Character blocking:** NPCs play specific animations on cue; the player is locked (Humanoid.WalkSpeed = 0) until the cutscene ends.
- **Skip:** always offer a skip button; long unskippable cutscenes lose players.

Part IX — Smoothness Levers (Making It Buttery)

The user said "make it as smooth and buttery as possible." This is the checklist of Roblox-specific performance and feel levers, sourced from create.roblox.com/docs/performance-optimization and devforum consensus.

9.1 The 60 FPS loop

- **RunService:BindToRenderStep(name, priority, fn)** for the combat loop, NOT Heartbeat. RenderStep runs before the frame renders, so visual state is always current.
- **dt accumulation:** accumulate real time, run fixed 60Hz logic steps. This decouples sim speed from framerate (a 144Hz monitor does not speed up the game).
- **Allocation-free hot path:** never Instance.new in the loop. Pool projectiles, particles, hitbox parts. Reuse.
- **Animation tracks:** :Play(0.1) fade-in/fade-out to avoid pops. Set track.Priority explicitly.

9.2 Hitbox smoothness (the "weighty feel" levers)

- **RaycastHitboxV4** (Swordphin, devforum.roblox.com/t/raycast-hitbox-401) — fires rays from each attachment per frame. Multiple attachments = swept volume. This is the community standard; do not roll your own.
- **HitStop** on heavy connects — freeze both fighters for 2-4 frames on a Smash hit. This is the single biggest "weight" lever. Street Fighter, Tekken, and BT3 all do this.
- **Camera shake** scaled to hit weight — light hits = none, Smash = small, Ultimate = large. Use a sine-decay shake, not random.
- **Hit flash** — the struck part flashes white for 1 frame. Pure visual, but it is what sells the connect.

9.3 Input smoothness (the "responsive feel" levers)

- **Input buffering:** if the player presses M1 during a move's recovery, queue it for 6 frames. If the move becomes cancelable in that window, the queued input fires. This is why good fighting games feel "snappy" — your input is never lost.
- **Just-pressed set:** track keys pressed THIS frame separately from keys held. The translator layer reads the just-pressed set for tap-counting (rush chain position).
- **Client-side move startup:** the client begins the move animation locally on input; the server confirms 50-100ms later. The visual is always in sync with input. If the server rejects (illegal state), the client cancels the animation with a brief red flash.

9.4 Network smoothness

- **Region-locked matchmaking** — only match within 120ms ping. This is the #1 lever.
- **Client-side hit detection** — attacker registers hit immediately, server validates. No round-trip for the visual.
- **Ping-bounded tolerance** — server accepts hits within 1.5x attacker ping.
- **Move prediction** — client predicts its own move startup; server corrects only if illegal.

Part X — Build Roadmap & Follow-Up Questions

10.1 The 4-phase build roadmap (revised for original IP + 6-char start)

Phase	Calendar	Team	Deliverable
1. Melee Prototype	~6 weeks	1-2	One character (Son Wukong base + SSJ), one arena, full Rush Chain + Smash + Vanish
2. Vertical Slice	~3 months	2-4	6 characters, RPG layer (stats + TP + Power Release), transformation wheel, first saga
3. Beta	~6 months	4-8	Full DBZ roster (98+), all races, all sagas through Buu, tournament mode, first raid boss
4. v1.0	~4 months	6-10	DBS god-tier content, space travel, full planet system, all raid bosses, console parity (P

10.2 Phase 1 detailed breakdown (the melee prototype — this is what we build first)

Week	Task	Output
1	Scaffold ReplicatedStorage/ServerScriptService structure. Build R15 hitboxing system. Verify Rig assets. Hitbox V4 imports.	
2	Implement SKILLS ModuleScript with the 6 core moves (GutPunch, BitingKneeSweep, HeavySmash, SmashCharge, Vanish, RushChain).	
3	Implement the state machine (idle/rushChain/smashCharge/hitstun/flight/backdrop/heavyWin/guard). Input translator for R15.	
4	Pursuit system (DragonHoming, VanishingStrike, LightningSlam). Keyboard + window + touch + tablet.	
5	Counter ladder (Guard, Super Perception, Revenge Counter, Super Absorb). First functional test: attacker + defender.	
6	Log-ratio damage formula. Power-level table for Son Wukong + Vegeta. Publish 1st stop, party shake, hit flash, input buffering.	

10.3 Follow-up questions for the user (please answer before we build)

#	Question	Why it matters
1	Confirm the 6 starting characters (Part I.2)? Or swap any? The animation + movelist scope for Phase 1.	Answers the animation + movelist scope for Phase 1.
2	Original-IP names: do you have your own, or should I propose a full needs-to-be settled before any public asset upload.	Should propose a full needs-to-be settled before any public asset upload.
3	Power levels (Part IV): any corrections before I lock it? The damage formula using PL = wrong feel.	Feedback on damage formula using PL = wrong feel.
4	God-tier handling (Part IV.5): OK with tier-relative multipliers (SSJ is 100, SSJ3 is 1000, SSJ4 is 10000, SSJ4+ is 100000)?	Multipliers (SSJ is 100, SSJ3 is 1000, SSJ4 is 10000, SSJ4+ is 100000)?
5	Transformation stacking (Part VII.2): is "SSB Kaioken sets the Kaioken limit" the design intent? Or different?	SSB Kaioken sets the Kaioken limit the design intent? Or different?
6	Monetization model: cosmetics-only (safest, Roblox-friendly, primarily for cosmetics (Rattle TP, Stone Plaque).	Roblox-friendly, primarily for cosmetics (Rattle TP, Stone Plaque).
7	Mobile: dedicated buttons (Part III.1) OK? Or do you have the input scheme in mind?	Have the input scheme in mind?
8	Should I start scaffolding Phase 1 in your connected Roblox game after you answer, or refine the plan further?	Should I start scaffolding Phase 1 in your connected Roblox game after you answer, or refine the plan further?

10.4 Sources (cited throughout)

#	Source	Used for
1	Bandai Namco, "DRAGON BALL: Sparking! ZERO — Combos and Feats" (bandainamco.com/en/2024/06/24/combos-and-feats/)	Stutes "Chain Limit Triangle spec" (Aug 2024) Combo = more damage
2	IGN, "Advanced Combat Tips - DRAGON BALL: Sparking! ZERO" (ign.com/articles/2024/08/24/advanced-combat-tips-dragon-ball-sparking-zero/)	Distant, long-range combos, etc. (Oct 2024) Ult spec, pursuit system
3	Bandai Namco Asia, "DRAGON BALL: Sparking! ZERO — The Beginning of the Journey" (bandainamco.com/en/2024/06/24/the-beginning-of-the-journey/)	Full roster, full chain spec, SSB, etc. (Oct 2024) Chain input
4	TheGamer, "Advanced Sparking Mode Techniques" (thegamer.com/sparking-mode-techniques/)	Sparking Mode resource interaction
5	Previous compendium: BT3 decompilation audit (sles_549.45, 292, 389, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000)	hitbox FUN_00232478, script VM a
6	Swordphin, "Raycast Hitbox 4.01: For all your melee needs!" (devforum.roblox.com/t/raycast-hitbox-401/3089136)	Community-owned raycast hitbox 4.01, updated, Sept-2021 to
7	"Client side or Server side hitboxes?" (devforum.roblox.com/t/client-side-or-server-side-hitboxes/3089136)	Client or server hitboxes (3089136) pattern
8	"Building Server-Truth Combat Systems in Roblox" (YouTube, search=)	Server-authority + lag-compensation pattern
9	gmmarket.me, "Roblox Replication — RemoteEvent vs RemoteFunction" (April 2026)	Server (April 2026) rule; anti-exploit
10	"Animation Tree Blueprint-Driven Runtime Animation Editing" (devforum.roblox.com/t/animation-tree-blueprint-driven-runtime-animation-editing/3089136)	Runtime animation module (Aug 2023)

#	Source	Used for
11	create.roblox.com/docs/animation	Roblox Animation Editor, Motor6D, AnimationPriority
12	"How to animate your tool using Motor6D" (devforum.roblox.com/t/Cu3026705)	Motor6D setup for tools/limbs
13	YouTube: "How To Create a Punch Animation In Roblox Studio", "How to make a Beam VFX in ROBLOX STUDIO! [Moon Ar"	Beam VFX, Beam VFX
14	"How would I make a beam VFX for a certain attack?" (devforum.roblox.com/t/294418) and YouTube, "Roblox Beam VFX 1ae"	Beam VFX, Beam VFX
15	kitsblox.com, "How to Make Smooth Camera Cutsscenes in Roblox" (Jan 2025), ripthor #174616 on DevForum, "Creating Multiple Cuts"	Camera Cutsscenes, Camera Cutsscenes
16	create.roblox.com/docs/performance-optimization	RenderStep, pooling, network ownership, performance lever
17	Sportskeeda "5 best Roblox games for Dragon Ball Z fans" (Jun 2023), Top-DBZ Roblox Games, Roblox DBZ Games, Jan 2024, red	Dragon Ball Z, Dragon Ball Z

This document is the working plan. Once the user answers the 8 follow-up questions in section 10.3, Phase 1 (the 6-week melee prototype) can begin in the connected Roblox Studio place. The melee system in Part III is the spec; the code in Part V is the implementation. Everything else (transformation wheel, RPG layer, content) builds on top of this melee foundation.